Recursion with ArrayList

1. Understanding Recursion with ArrayList

What is Recursion?

Recursion is a programming technique where a method calls itself to solve smaller versions of the same problem. Every recursive method needs:

- **Base case**: A condition that stops the recursion
- **Recursive case**: The method calling itself with modified parameters

Working with ArrayList in Recursion

ArrayList is a dynamic array that can grow or shrink during runtime. When using recursion with ArrayList, we can:

- Process elements one by one
- Modify the list during recursive calls
- Build new lists based on conditions

Basic Structure of Recursive Methods with ArrayList

```java
public returnType recursiveMethod(ArrayList<DataType> list, int index) {
    // Base case
    if (index >= list.size()) {
        return baseValue;
    }

    // Recursive case
    // Process current element
    // Make recursive call with modified parameters
    return recursiveMethod(list, index + 1);
}
```

Example 1: Finding Sum of All Elements

```java
import java.util.ArrayList;

public class RecursionExample {
    public static int findSum(ArrayList<Integer> numbers, int index) {
        // Base case: if we've processed all elements
        if (index >= numbers.size()) {
            return 0;
        }

        // Recursive case: add current element to sum of remaining elements
        return numbers.get(index) + findSum(numbers, index + 1);
    }

    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        nums.add(5);
        nums.add(10);
        nums.add(15);

        int total = findSum(nums, 0);
        System.out.println("Sum: " + total); // Output: Sum: 30
    }
}
```

Example 2: Finding Maximum Element

```java
public static int findMax(ArrayList<Integer> numbers, int index) {
    // Base case: if only one element left
    if (index == numbers.size() - 1) {
        return numbers.get(index);
    }

    // Recursive case: compare current element with max of remaining elements
    int maxOfRest = findMax(numbers, index + 1);
    return Math.max(numbers.get(index), maxOfRest);
}
```

Example 3: Counting Occurrences of a Value

```java
public static int countOccurrences(ArrayList<String> words, String target, int index) {
    // Base case: processed all elements
    if (index >= words.size()) {
        return 0;
    }

    // Recursive case: check current element and add count from rest
    int currentCount = words.get(index).equals(target) ? 1 : 0;
    return currentCount + countOccurrences(words, target, index + 1);
}
```

Example 4: Reversing an ArrayList

```java
public static void reverseList(ArrayList<Integer> list, int start, int end) {
    // Base case: pointers meet or cross
    if (start >= end) {
        return;
    }

    // Swap elements at start and end positions
    int temp = list.get(start);
    list.set(start, list.get(end));
    list.set(end, temp);

    // Recursive case: move pointers inward
    reverseList(list, start + 1, end - 1);
}
```

Example 5: Filtering Elements (Creating New ArrayList)

```java
public static ArrayList<Integer> filterEven(ArrayList<Integer> original, int index, ArrayList<I
    // Base case: processed all elements
    if (index >= original.size()) {
        return result;
    }

    // If current element is even, add it to result
    if (original.get(index) % 2 == 0) {
        result.add(original.get(index));
    }

    // Recursive case: process next element
    return filterEven(original, index + 1, result);
}
```

2. Practice Questions

Question 1: Binary Search in Sorted ArrayList

```java
public static int binarySearch(ArrayList<Integer> sortedList, int target, int left, int right)
    // Base case: element not found
    if (left > right) {
        return -1;
    }

    int mid = (left + right) / 2;

    // Element found
    if (sortedList.get(mid) == target) {
        return mid;
    }

    // Search in left half
    if (target < sortedList.get(mid)) {
        return binarySearch(sortedList, target, left, mid - 1);
    }

    // Search in right half
    return binarySearch(sortedList, target, mid + 1, right);
}
```

Question 2: Palindrome Check

```java
public static boolean isPalindrome(ArrayList<Character> chars, int start, int end) {
    // Base case: single character or empty
    if (start >= end) {
        return true;
    }

    // Check if characters at both ends match
    if (!chars.get(start).equals(chars.get(end))) {
        return false;
    }

    // Recursively check inner characters
    return isPalindrome(chars, start + 1, end - 1);
}
```

Question 3: Finding All Indices of a Value

```java
public static ArrayList<Integer> findAllIndices(ArrayList<String> list, String target, int inde
    // Base case: processed all elements
    if (index >= list.size()) {
        return indices;
    }

    // If current element matches target, add index
    if (list.get(index).equals(target)) {
        indices.add(index);
    }

    // Recursive case: check next element
    return findAllIndices(list, target, index + 1, indices);
}
```

Quiz Questions

1. What are the two essential components of any recursive method? a) Loop and condition b) Base case and recursive case c) ArrayList and index d) Parameters and return type

Answer: b) Base case and recursive case

2. In the following recursive method, what would be the output for an ArrayList containing [2, 4, 6]?

```java
public static int mystery(ArrayList<Integer> list, int index) {
    if (index >= list.size()) return 1;
    return list.get(index) * mystery(list, index + 1);
}
```

a) 12
b) 48
c) 24
d) 6

Answer: b) 48

3. Which of the following is the correct base case for finding the minimum element in an ArrayList using recursion? a) `if (index == 0)` b) `if (index >= list.size())` c) `if (index == list.size() - 1)` d) `if (list.isEmpty())`

Answer: c) if (index == list.size() - 1)

4. What happens if a recursive method doesn't have a proper base case? a) The program runs faster b) Stack overflow error occurs c) The method returns null d) Compilation error

Answer: b) Stack overflow error occurs

5. When reversing an ArrayList recursively by swapping elements from both ends, what should be the base case? a) `if (start == 0)` b) `if (end == list.size())` c) `if (start >= end)` d) `if (list.isEmpty())`

Answer: c) if (start >= end)

Practice Coding Questions

1. Write a recursive method `removeAllOccurrences` that takes an ArrayList of integers and a target value, and removes all occurrences of that target value from the list. The method should modify the original list.

2. Create a recursive method `mergeSortedLists` that takes two sorted ArrayLists of integers and returns a new ArrayList containing all elements from both lists in sorted order.

3. Implement a recursive method `calculateAverage` that finds the average of all elements in an ArrayList of doubles. The method should return the average as a double value.

4. Write a recursive method `hasConsecutiveDuplicates` that checks if an ArrayList contains any two consecutive elements that are equal. Return true if duplicates exist, false otherwise.

5. Create a recursive method `flattenNestedList` that takes an ArrayList containing other ArrayLists of integers and returns a single flattened ArrayList containing all the integers in order.